

DATA ENGINEERING COURSE

OOP in Python

Inheritance · Encapsulation · File Processing

Class Notes

Instructor: Tanishq Tyagi

Data Engineering Course

Table of Contents

1.	Introduction to Inheritance	3
2.	Parent Class & Child Class	4
3.	Method Overriding	6
4.	Encapsulation	8
5.	Getter & Setter Methods	10
6.	FileProcessor Project	12
7.	CSVProcessor & JSONProcessor	14
8.	Private Variables & Access Control	16
9.	Practical Examples	18
10.	Real-World Data Engineering Perspective	20
11.	Summary & Quick Revision Sheet	22
12.	General Interview Questions	24
13.	Data Engineer Interview Preparation	28

Session Overview

This session covers core Object-Oriented Programming (OOP) concepts as applied in Python, with a specific focus on their relevance to Data Engineering workflows. We explore **Inheritance** — how classes reuse and extend behaviour — **Method Overriding**, **Encapsulation** with private attributes, and Getter/Setter patterns. We then apply these concepts in a realistic **FileProcessor** project that models how data engineers handle CSV and JSON files in production pipelines.

Learning Objectives

By the end of this session, students will be able to:

- Define and implement **Inheritance** in Python using parent and child classes.
- Explain the concept of **Method Overriding** and when to apply it.
- Apply **Encapsulation** to protect class data using private attributes.
- Implement **Getter and Setter** methods with validation logic.
- Build a real-world **FileProcessor** hierarchy using OOP principles.
- Distinguish between public, protected, and private access levels in Python.
- Apply OOP concepts to design extensible data pipeline components.
- Prepare for OOP and Data Engineering interview questions.

Part 1 — Introduction to Inheritance

Definition

Inheritance is an OOP mechanism that allows one class (the *child class*) to acquire the properties and behaviour of another class (the *parent class*). It promotes **code reuse**, reduces redundancy, and models real-world relationships.

Why Inheritance Matters

- Eliminates duplicate code — write logic once, reuse everywhere.
- Models real-world hierarchies (Animal → Dog, FileProcessor → CsvProcessor).
- Makes codebases easier to maintain and extend.
- Enables polymorphism — different classes can be used interchangeably.

- **Key Terminology** • **Parent class** (also: base class, super class) — the class being inherited from.
- **Child class** (also: sub class, derived class) — the class that inherits.

Syntax

```
class ParentClass:

    # parent attributes and methods

    pass

class ChildClass(ParentClass):    # inherit from ParentClass

    # child-specific attributes and methods

    pass
```

Types of Inheritance

- **Single Inheritance** — one child, one parent. (Most common)
- **Multi-level Inheritance** — chain: A → B → C.
- **Multiple Inheritance** — one child, many parents (use carefully).
- **Hierarchical Inheritance** — one parent, many children.

Common Mistakes

- Forgetting to call `super().__init__()` when the parent has an `__init__`.
- Creating unnecessarily deep inheritance chains (prefer composition for complex cases).
- Assuming the child always needs to override every parent method.


```
labra_dog.sound()    # Output: Bark!! (inherited from Dog)
```

■ Notice that **Cat** uses `pass` — it inherits `sound()` from `Animal` without modification. **LabraDog** inherits from `Dog` which itself inherits from `Animal`, demonstrating multi-level inheritance.

How Python Resolves Methods — MRO

Python uses the **Method Resolution Order (MRO)** to find which class's method to call. It searches the class hierarchy from child to parent, left to right.

```
print(LabraDog.__mro__)

# Output:

# (<class '__main__.LabraDog'>, <class '__main__.Dog'>,
#  <class '__main__.Animal'>, <class 'object'>)
```

Part 3 — Method Overriding

Definition

Method Overriding occurs when a child class provides its **own implementation** of a method that is already defined in the parent class. The child's version replaces the parent's version for instances of the child class.

When to Override

- When the child class needs different behaviour from the parent.
- When the parent provides a *default* or *abstract* behaviour.
- When implementing polymorphism — treating different objects the same way.

Abstract-style Methods with `NotImplementedError`

A common Python pattern is to raise `NotImplementedError` in the parent to force child classes to provide their own implementation:

```
class FileProcessor:

    def read(self):

        raise NotImplementedError("Child class must implement read()")
```

■ This pattern is Python's way of creating abstract methods without the `abc` module. If a child class forgets to override `read()`, calling it will raise a clear error.

Calling the Parent Method — `super()`

```
class Animal:

    def sound(self):

        print("Animal sound")

class Dog(Animal):

    def sound(self):

        super().sound()    # call parent's version first

        print("Bark!!")

dog = Dog()

dog.sound()
```

```
# Output:  
  
# Animal sound  
  
# Bark!!
```

Best Practices

- Use `super()` to extend parent behaviour rather than completely replace it when appropriate.
- Keep method signatures consistent with the parent class.
- Document *why* the method is being overridden.

Part 4 — Encapsulation

Definition

Encapsulation means **hiding internal implementation details** and controlling how data is accessed and modified. It is one of the four pillars of OOP.

Why Encapsulation?

Consider this unprotected example from class:

```
balance = 100000

# Any code anywhere can overwrite it:

balance = -1000    # No validation, no control!
```

With encapsulation, we move the data *inside* a class and control access through methods — preventing accidental or malicious modification.

Access Levels in Python

Level	Syntax	Access From	Example
Public	name	Anywhere	self.name
Protected	_name	Class + subclasses	self._name
Private	__name	Only within the class	self.__balance

■■ Python does not truly prevent access to private variables — it uses **name mangling**. `__balance` becomes `_BankAccount__balance`. This is a convention, not hard enforcement. The double underscore signals '*do not touch this from outside the class*'.

Part 5 — Getter & Setter Methods

Definition

- **Getter** — a method that *reads* (returns) a private attribute.
- **Setter** — a method that *writes* (updates) a private attribute, usually after validation.

BankAccount Example (from class)

[illegible]

Using the Class

```
mohit_account = BankAccount("Mohit Pandey", 100000)
```

```
print(mohit_account.name)                # Mohit Pandey

print(mohit_account.get_account_balance()) # 100000

mohit_account.set_account_balance(1000)

print(mohit_account.get_account_balance()) # 1000

# Trying to set a negative balance:

mohit_account.set_account_balance(-500)   # Raises InvalidAmountException
```

Line-by-Line Explanation

- `self.__account_balance` — private attribute; cannot be accessed directly outside the class.
- `set_account_balance()` — validates input before updating; negative values raise a custom exception.
- `get_account_balance()` — simulates authentication by checking the account owner's name.
- Custom exceptions (`InvalidAmountException`, `InvalidUserError`) provide clear error messages.

■ **Best Practice:** Always validate data inside setters. Never let invalid state enter your objects. This is especially important in data pipelines where bad data can propagate silently.

Part 6 — FileProcessor Project

The FileProcessor project brings all OOP concepts together in a realistic data engineering scenario. It models how different file types are processed in a data pipeline.

Project Architecture

```
FileProcessor  (Parent – defines the contract)
■
■■■ CsvProcessor  (Child – processes CSV files)
■■■ JsonProcessor (Child – processes JSON files)
■■■ TextFileProcessor (Child – inherits, no override yet)
```

FileProcessor — Parent Class

```
import os

class FileProcessor:

    def validate_file(self, file_path):

        if not os.path.exists(file_path):

            raise FileNotFoundError(f"{file_path} not found!")

    def log(self, message):

        print(f"[LOG] {message}")

    def read(self):

        raise NotImplementedError("Child class must implement read()")
```

Line-by-Line Explanation

- `validate_file()` — reusable utility shared by all child classes; checks file existence.
- `log()` — shared logging utility; all processors can call this without duplicating code.
- `read()` — defines the *contract*: every child MUST provide its own read logic.

Child Classes

```
class CsvProcessor(FileProcessor):
```

```
def read(self):  
    print("Reading a CSV file!")  
  
class JsonProcessor(FileProcessor):  
    def read(self):  
        self.validate_file('data.json')    # reuse parent utility  
        print("Reading a JSON file!")  
  
class TextFileProcessor(FileProcessor):  
    pass    # Will raise NotImplementedError if read() is called
```

Testing the Hierarchy

```
csv_processor = CsvProcessor()  
csv_processor.read()    # Output: Reading a CSV file!  
  
json_processor = JsonProcessor()  
json_processor.read()    # Output: Reading a JSON file!  
  
txt_processor = TextFileProcessor()  
txt_processor.read()    # Raises: NotImplementedError
```

■ This pattern is used in every major data framework. Apache Spark, Airflow, SQLAlchemy — all define base classes with abstract methods that subclasses must implement.

Part 7 — CSVProcessor & JSONProcessor

Let's extend the FileProcessor project into a fully functional implementation that reads actual CSV and JSON files.

Extended CsvProcessor

```
import csv

class CsvProcessor(FileProcessor):

    def __init__(self, file_path):

        self.file_path = file_path

    def read(self):

        self.validate_file(self.file_path)

        self.log(f'Reading CSV: {self.file_path}')

        with open(self.file_path, 'r') as f:

            reader = csv.DictReader(f)

            for row in reader:

                print(row)
```

Extended JsonProcessor

```
import json

class JsonProcessor(FileProcessor):

    def __init__(self, file_path):

        self.file_path = file_path

    def read(self):

        self.validate_file(self.file_path)

        self.log(f'Reading JSON: {self.file_path}')

        with open(self.file_path, 'r') as f:
```

```
data = json.load(f)

print(data)
```

Key Observations

- Both processors reuse `validate_file()` and `log()` from the parent.
- Each processor only implements *what is different* — the file-reading logic.
- Adding a new file type (e.g., `ParquetProcessor`) requires only a new child class.

■ **Real-World Note:** In production ETL pipelines, this exact pattern is used. A base `DataReader` class defines the interface; specific implementations handle S3, GCS, HDFS, local files, databases, etc.

Part 8 — Private Variables & Access Control

Name Mangling

When you prefix an attribute with double underscore (`__`), Python applies **name mangling**:

```
class BankAccount:

    def __init__(self):

        self.__balance = 1000    # stored as _BankAccount__balance

acc = BankAccount()

# print(acc.__balance)           # AttributeError

print(acc._BankAccount__balance) # 1000 (name mangling exposed)
```

Protected Attributes

Single underscore (`_name`) is a *convention* meaning 'internal use', but it is still accessible:

```
class A:

    def __init__(self, name):

        self._name = name    # protected – accessible but 'please don't'

a = A('Alice')

print(a._name)    # Alice – works, but not recommended from outside
```

Summary: Access Levels

- **Public** (`name`): accessible from anywhere.
- **Protected** (`_name`): accessible, but convention says 'internal'.
- **Private** (`__name`): name-mangled; not directly accessible outside the class.

■■ Python's philosophy: 'We're all consenting adults.' Access modifiers are enforced by convention, not the language. Use them consistently to write clean, maintainable code.

Part 9 — Practical Examples

Example 1: Shape Hierarchy

```
class Shape:

    def area(self):

        raise NotImplementedError("Implement area() in child class")

class Rectangle(Shape):

    def __init__(self, width, height):

        self.width = width

        self.height = height

    def area(self):

        return self.width * self.height

class Circle(Shape):

    def __init__(self, radius):

        self.radius = radius

    def area(self):

        return 3.14159 * self.radius ** 2

shapes = [Rectangle(4, 5), Circle(3)]

for shape in shapes:

    print(f'Area: {shape.area()}')

# Area: 20

# Area: 28.27...
```

Example 2: Employee with Encapsulation

```
class Employee:

    def __init__(self, name, salary):

        self.name = name
```

```
        self.__salary = salary

    def get_salary(self):

        return self.__salary

    def set_salary(self, salary):

        if salary < 0:

            raise ValueError("Salary cannot be negative!")

        self.__salary = salary

emp = Employee("Alice", 50000)

print(emp.get_salary())    # 50000

emp.set_salary(60000)

print(emp.get_salary())    # 60000
```

Part 10 — Real-World Data Engineering Perspective

Inheritance in Data Pipelines

- Airflow **BaseOperator** → custom operators inherit `execute()` logic.
- SQLAlchemy **Base** → ORM models inherit database mapping.
- PySpark **Transformer** pattern — each stage inherits a `transform()` interface.
- Great Expectations → custom expectation classes inherit from base classes.

Encapsulation in Production Systems

- Database connection strings — never expose directly; always behind getters with auth.
- API keys and secrets — stored in private attributes, never logged or printed.
- Internal pipeline state — private counters, checkpoints, retry counts.

FileProcessor in ETL

The FileProcessor pattern maps directly to real ETL architecture:

```
# Production-style ETL processor hierarchy

class DataReader:

    def read(self): raise NotImplementedError

    def validate(self): raise NotImplementedError

class S3CsvReader(DataReader):

    def read(self): ...

class GCSJsonReader(DataReader):

    def read(self): ...

class KafkaStreamReader(DataReader):

    def read(self): ...
```

Scalability Considerations

- Inheritance hierarchies should be **shallow** in production (max 3-4 levels).
- Prefer **composition over inheritance** for complex behaviours.
- Abstract base classes (`abc.ABC`) enforce contracts in production systems.
- Use dependency injection — pass processor instances rather than hard-coding classes.

Part 11 — Summary & Quick Revision Sheet

Key Concepts at a Glance

Concept	One-Line Definition	Syntax
Inheritance	Child acquires parent behaviour	<code>class Child(Parent):</code>
Method Override	Child redefines parent method	<code>def method(self):</code>
<code>super()</code>	Calls parent's version of a method	<code>super().method()</code>
Encapsulation	Hides data, controls access	<code>self.__attr</code>
Getter	Reads a private attribute	<code>def get_x(self):</code>
Setter	Writes with validation	<code>def set_x(self, v):</code>
Name Mangling	<code>__attr</code> → <code>_ClassName__attr</code>	<code>obj._Class__attr</code>
<code>NotImplementedError</code>	Forces child to override	<code>raise NotImplementedError()</code>

Quick Revision — Cheat Sheet

```
# Inheritance

class Child(Parent): pass


# Multi-level

class GrandChild(Child): pass


# Method Override

class Child(Parent):

    def method(self):

        super().method()    # optional: call parent

        # child-specific logic


# Encapsulation

class MyClass:

    def __init__(self):

        self.public    = 'visible everywhere'
```

```
        self._protected = 'visible, but internal'

        self.__private = 'name-mangled'

    def get_private(self): return self.__private

    def set_private(self, v): self.__private = v

# Force override pattern

class Base:

    def process(self):

        raise NotImplementedError('Override me!')
```

Part 12 — General Interview Questions

A. Beginner Level

Q1. What is inheritance in Python?

Inheritance is a mechanism where a child class acquires the attributes and methods of a parent class. It promotes code reuse and models 'is-a' relationships. Syntax: `class Child(Parent):`

Q2. What is a parent class and a child class?

A parent class (base/super class) defines common attributes and methods. A child class (derived/sub class) inherits from the parent and can extend or override its behaviour.

Q3. What is method overriding?

Method overriding occurs when a child class defines a method with the same name as one in the parent. Python uses the child's version when called on an instance of the child.

Q4. What is encapsulation?

Encapsulation hides internal data and implementation details. Access is controlled through methods (getters/setters) to protect data from accidental modification.

Q5. What is a getter method?

A getter method returns the value of a private attribute. Example: `def get_balance(self): return self.__balance`

Q6. What is a setter method?

A setter method updates a private attribute, typically after validating the new value. Example: `def set_balance(self, amount): if amount >= 0: self.__balance = amount`

Q7. How do you make an attribute private in Python?

Prefix the attribute name with two underscores: `self.__attribute`. Python applies name mangling, storing it as `_ClassName__attribute`.

Q8. What is `super()` used for?

`super()` refers to the parent class. It is used to call the parent's methods, especially `__init__()`, from within a child class.

Q9. What is `NotImplementedError`?

Raising `NotImplementedError` in a parent class method signals that child classes must provide their own implementation. It is Python's way of creating abstract-style methods.

Q10. What is the difference between public, protected, and private attributes?

Public (attr): accessible anywhere. Protected (`_attr`): accessible but conventionally internal. Private (`__attr`): name-mangled; not directly accessible outside the class.

Q11. Can a child class access private attributes of the parent?

No, not directly. Private attributes are name-mangled. The child must use the parent's getter/setter methods to access them.

Q12. What is multi-level inheritance?

When a class inherits from another child class: $A \rightarrow B \rightarrow C$. Example from class: `Animal` $\rightarrow$ `Dog` $\rightarrow$ `LabraDog`.

B. Intermediate Level

Q1. What is MRO (Method Resolution Order)?

MRO defines the order in which Python searches classes when resolving a method. Python uses the C3 Linearization algorithm. You can inspect it with `ClassName.__mro__`.

Q2. What is the difference between name mangling and access modifiers in other languages?

In Java/C++, private means truly inaccessible. In Python, private (`__attr`) uses name mangling — the attribute is renamed to `_Class__attr`. It can still be accessed via the mangled name, so Python relies on developer convention.

Q3. When would you use composition instead of inheritance?

Use composition when the 'has-a' relationship is more appropriate than 'is-a'. Example: a Car 'has-a' Engine (composition) vs a Dog 'is-a' Animal (inheritance). Composition is also preferred when the hierarchy would become deep or complex.

Q4. What happens if a child class doesn't override a method that raises `NotImplementedError`?

Calling that method on the child instance will raise a `NotImplementedError` at runtime, clearly indicating that the child class has not fulfilled the required contract.

Q5. How does Python's `@property` decorator relate to getters and setters?

`@property` provides a more Pythonic way to define getters. Combined with `@attr.setter`, it allows attribute-style access while still running validation logic.

Q6. What is the difference between `isinstance()` and `type()`?

`isinstance(obj, ClassName)` returns `True` if `obj` is an instance of `ClassName` or any subclass. `type(obj) == ClassName` is strict — `True` only for the exact class, not subclasses. `isinstance()` is preferred for OOP code.

Q7. Can you inherit from multiple parents in Python?

Yes. Python supports multiple inheritance: `class C(A, B):`. MRO determines method resolution order. Use with care to avoid the 'diamond problem'.

Q8. What are abstract base classes (ABCs)?

Python's `abc` module provides `ABC` and `@abstractmethod`. A class with abstract methods cannot be instantiated until all abstract methods are implemented. This is a stricter version of the `NotImplementedError` pattern.

Q9. What is duck typing and how does it relate to inheritance?

Duck typing means Python checks whether an object has the required methods/attributes, not whether it inherits from a specific class. This allows objects from unrelated hierarchies to be used interchangeably.

Q10. How do custom exceptions work in Python?

Custom exceptions are classes that inherit from the built-in `Exception` class. Example: `class InvalidAmountException(Exception): pass`. They can be raised and caught like built-in exceptions.

Q11. Explain the `__init__` and `__str__` dunder methods.

`__init__` is the constructor, called when an object is created. `__str__` is called by `print()` and returns a human-readable string. Dunder (double underscore) methods are Python's special/magic methods.

Q12. What is the Liskov Substitution Principle?

Any child class should be usable wherever the parent class is expected, without breaking the program. This means child classes should not weaken preconditions or strengthen postconditions of parent methods.

Part 13 — Data Engineer Interview Preparation

■ This section prepares you for real Data Engineering interviews. Questions focus on how OOP concepts are applied in production data systems, pipelines, and ETL/ELT workflows.

DE-Q1. How is inheritance used in data pipeline frameworks?

Frameworks like Apache Airflow, Luigi, and Prefect use inheritance extensively. Airflow's `BaseOperator` defines the contract (`execute()` method), and engineers create custom operators by inheriting from it. For example, a `S3ToRedshiftOperator` inherits from `BaseOperator` and overrides `execute()`. This allows the scheduler to treat all operators uniformly while each implements different logic.

DE-Q2. How would you design a file processor hierarchy for an ETL pipeline?

You define a base `DataReader` class with abstract methods like `read()`, `validate()`, and `log()`. Each file format (CSV, JSON, Parquet, Avro) gets its own class inheriting from `DataReader`. This allows the pipeline orchestrator to call `reader.read()` uniformly regardless of format. Adding support for a new format only requires a new child class — open-closed principle.

DE-Q3. Why is encapsulation important in production data systems?

In production, careless data access leads to bugs that are hard to trace. Encapsulation ensures that sensitive data (credentials, connection strings, API keys) are never exposed or accidentally overwritten. Setters enforce validation, preventing invalid states from propagating through the pipeline.

DE-Q4. How are custom exceptions used in ETL pipelines?

Custom exceptions like `DataValidationError`, `SchemaError`, `FileNotFoundError` allow the pipeline to handle different failure modes differently. For example, a `DataValidationError` might trigger data quarantine, while a `ConnectionError` triggers a retry with exponential backoff. Generic Exception catches are avoided in production.

DE-Q5. What is the 'template method' design pattern and how does it apply to data engineering?

The template method pattern defines the skeleton of an algorithm in a base class, with specific steps deferred to subclasses. This is exactly the `FileProcessor` pattern: the base class defines `validate()` and `log()` (invariant steps), while `read()` is deferred. ETL pipelines commonly use this pattern for `extract` → `transform` → `load` sequences.

DE-Q6. How does Python's name mangling affect pipeline debugging?

Private attributes are name-mangled, making them harder to inspect during debugging. A common production practice is to expose private state through controlled getter methods that also log access. Some teams use `@property` for a cleaner interface. Never rely on name-mangled attributes from outside the class — they may change.

DE-Q7. When would you choose multiple inheritance in a data engineering context?

Rarely. Multiple inheritance can introduce the diamond problem and make code hard to maintain. A common safe use is mixin classes: a `LoggingMixin` adds logging behaviour, a `ValidationMixin` adds

validation. Your class inherits from `DataReader` + `LoggingMixin` + `ValidationMixin`. Mixins should have no state and no `__init__`.

DE-Q8. How do abstract base classes improve data pipeline reliability?

Using Python's `abc.ABC` and `@abstractmethod`, you can enforce that any class claiming to be a `DataReader` must implement `read()`. Attempting to instantiate an incomplete implementation raises a `TypeError` at startup, catching bugs before the pipeline runs. This is preferable to discovering missing methods at runtime.

DE-Q9. How would you implement validation in a data model class?

Through setter methods with explicit validation logic and custom exceptions. For example, a `SalesRecord` class might validate that `quantity >= 0`, `price > 0`, and `date` is not in the future. The `__init__` calls setters to ensure all initial values are also validated. This is a critical data quality practice in pipelines.

DE-Q10. Explain how the `FileProcessor` pattern scales in a distributed environment.

In a distributed pipeline (Spark, Flink), the `FileProcessor` hierarchy translates to `DataSource` classes. Each `DataSource` implements `read()` which can return a Spark `DataFrame`. The orchestration layer calls `source.read() → transform() → sink.write()` uniformly. This abstraction allows switching from local CSV to S3 Parquet with zero changes to pipeline logic — only the `DataSource` class changes.

DE-Q11. What is the difference between is-a and has-a relationships in pipeline design?

Is-a (inheritance): `CsvProcessor` IS-A `FileProcessor`. Has-a (composition): `Pipeline` HAS-A `FileProcessor`. In pipelines, composition is often preferred. A `Pipeline` class holds a reference to a reader, transformer, and writer object. This makes pipelines more flexible and testable because any component can be swapped without changing the `Pipeline` class itself.

DE-Q12. How do you handle missing method implementations in a plugin-based pipeline?

Use abstract base classes to define the plugin interface. All plugins must implement required methods. At plugin registration time, Python checks the interface. Alternatively, use duck typing with `try/except AttributeError` when calling plugin methods. Production systems often combine both: ABC for core methods, duck typing for optional features.

DE-Q13. How would you implement access control for a multi-tenant data platform?

Each tenant's data access class would use private attributes for credentials and connection strings. Getter methods perform authentication checks before returning data. A permission decorator pattern is applied on top. Private attributes ensure credentials cannot leak between tenant contexts.

DE-Q14. What are the performance implications of deep inheritance hierarchies?

Deep hierarchies increase method lookup time (MRO traversal), though in practice this is negligible compared to I/O costs in data pipelines. The real cost is maintainability — deep hierarchies are harder to understand and debug. In data engineering, keep hierarchies shallow (2-3 levels) and prefer composition for complex behaviours.

DE-Q15. How does encapsulation contribute to pipeline testability?

Encapsulated classes with clear interfaces are easy to mock in unit tests. Instead of a real `CsvProcessor` reading from disk, a mock object with the same getter/setter interface can return controlled test data. This is the foundation of dependency injection and test-driven development in data engineering projects.
